# UV & marimo

## The new Python tools you should know about...

Jean-Luc CHARLES

Jean-Luc.Charles@mailo.com

"RDVS Numériques" du département DuMAS de l'I2M
5 mars 2026

## Summary

- Why use a **PVE** (Python Virtual Environment) for your Python projects?
- **UV**: An extremely fast Python package and project manager, written in Rust
- **marimo**: Python notebooks to replace Jupyter notebooks?

## Summary

- Why use a **PVE** (Python Virtual Environment) for your Python projects?
- **UV**: An extremely fast Python package and project manager, written in Rust
- **marimo**: Python notebooks to replace Jupyter notebooks?

## Ressources

- GitHub repository for the PDF slides and the notebooks: github.com/cjlux/marimo_notebooks

# Who am I

## Season 1

- Industrial career of ~10 years
  Signal Processing at Thomson-SINTRA
  OO design & C++ development at IT companies (Capgemini...)
- I joined the I2M lab of ENSAM (Bordeaux) as lecturer
  I led the design and development of the GranOO workbench (*DEM computing workbench*)
  I taught Scientific & OO programming in Python at ENSAM
- I started to work on AI & Deep Learning in 2015.

# Who am I

## Season 1

- Industrial career of ~10 years
  Signal Processing at Thomson-SINTRA
  OO design & C++ development at IT companies (Capgemini...)
- I joined the I2M lab of ENSAM (Bordeaux) as lecturer
  I led the design and development of the GranOO workbench (*DEM computing workbench*)
  I taught Scientific & OO programming in Python at ENSAM
- I started to work on AI & Deep Learning in 2015.

## Season 2

- Since Sept. 2023 $\rightsquigarrow$ still working on *AI / Data processing* with Python as a freelancer.

# Python Virtual Environment (PVE)

## Interest of a PVE

State of the art in Python programming:
$\rightsquigarrow$ developing each project within a **PVE** providing an **isolated**, **dedicated**, **persistent** environment:

- containing a specific installation of Python
- **independent** of other Python installations likely to coexist on the same machine
- **independent** of computer updates.

# Python Virtual Environment (PVE)

## Interest of a PVE

State of the art in Python programming:
$\rightsquigarrow$ developing each project within a **PVE** providing an **isolated**, **dedicated**, **persistent** environment:

- containing a specific installation of Python
- **independent** of other Python installations likely to coexist on the same machine
- **independent** of computer updates.

# Python Virtual Environment (PVE)

## Interest of a PVE

State of the art in Python programming:
$\rightsquigarrow$ developing each project within a **PVE** providing an **isolated**, **dedicated**, **persistent** environment:

- containing a specific installation of Python
- **independent** of other Python installations likely to coexist on the same machine
- **independent** of computer updates.

# Python Virtual Environment (PVE)

## Interest of a PVE

State of the art in Python programming:
$\rightsquigarrow$ developing each project within a **PVE** providing an **isolated**, **dedicated**, **persistent** environment:

- containing a specific installation of Python
- **independent** of other Python installations likely to coexist on the same machine
- **independent** of computer updates.

# Python Virtual Environment (PVE)

## A simple implementation...

A PVE is based on a **dedicated disk tree** that houses:

- the desired version of the Python interpreter
- the specific versions of the Python modules for the project
- the specific associated tools (binaries, scripts...).

# Python Virtual Environment (PVE)

## A simple implementation...

A PVE is based on a **dedicated disk tree** that houses:

- the desired version of the Python interpreter
- the specific versions of the Python modules for the project
- the specific associated tools (binaries, scripts...).

# Python Virtual Environment (PVE)

## A simple implementation...

A PVE is based on a **dedicated disk tree** that houses:

- the desired version of the Python interpreter
- the specific versions of the Python modules for the project
- the specific associated tools (binaries, scripts...).

# Python Virtual Environment (PVE)

## A simple implementation...

A PVE is based on a **dedicated disk tree** that houses:

- the desired version of the Python interpreter
- the specific versions of the Python modules for the project
- the specific associated tools (binaries, scripts...).

# Python Virtual Environment (PVE)

## A simple implementation...

A PVE is based on a **dedicated disk tree** that houses:

- the desired version of the Python interpreter
- the specific versions of the Python modules for the project
- the specific associated tools (binaries, scripts...).

## Easy to use...

You can **create**, **duplicate**, **delete** and **recreate** a PVE easily, without impacting other Python installations possibly present on your computer.

# Python Virtual Environment (PVE)

## Commonly used tools to manage Python project within a PVE

- the **conda** command, available if you have installed miniconda (or Anaconda) on your computer
- the **venv** Python module [docs.python.org/3/library/venv.html]
- the **poetry** tool [poetry.org]

...

# Python Virtual Environment (PVE)

## Commonly used tools to manage Python project within a PVE

- the **conda** command, available if you have installed miniconda (or Anaconda) on your computer
- the **venv** Python module [docs.python.org/3/library/venv.html]
- the **poetry** tool [poetry.org]

...

# Python Virtual Environment (PVE)

## Commonly used tools to manage Python project within a PVE

- the **conda** command, available if you have installed miniconda (or Anaconda) on your computer
- the **venv** Python module [docs.python.org/3/library/venv.html]
- the **poetry** tool [poetry.org]

...

# Python Virtual Environment (PVE)

## Commonly used tools to manage Python project within a PVE

- the **conda** command, available if you have installed miniconda (or Anaconda) on your computer
- the **venv** Python module [docs.python.org/3/library/venv.html]
- the **poetry** tool [poetry.org]

...

# Python Virtual Environment (PVE)

## Commonly used tools to manage Python project within a PVE

- the **conda** command, available if you have installed miniconda (or Anaconda) on your computer
- the **venv** Python module [docs.python.org/3/library/venv.html]
- the **poetry** tool [poetry.org]

...

## The new way...

- Recently a new tool has been unveiled to manage Python project within a PVE : **uv**
- It is considerably **faster** and **easier** to use...

# Python Virtual Environment (PVE)

## Commonly used tools to manage Python project within a PVE

- the **conda** command, available if you have installed miniconda (or Anaconda) on your computer
- the **venv** Python module [docs.python.org/3/library/venv.html]
- the **poetry** tool [poetry.org]

...

## The new way...

- Recently a new tool has been unveiled to manage Python project within a PVE : **uv**
- It is considerably **faster** and **easier** to use...

# Python Virtual Environment (PVE)

## Commonly used tools to manage Python project within a PVE

- the **conda** command, available if you have installed miniconda (or Anaconda) on your computer
- the **venv** Python module [docs.python.org/3/library/venv.html]
- the **poetry** tool [poetry.org]

...

## The new way...

- Recently a new tool has been unveiled to manage Python project within a PVE : **uv**
- It is considerably **faster** and **easier** to use...

# UV ⇝ the new Python project/PVE manager

## 😀 UV from Astral

**docs.astral.sh/uv/**

- For more than a decade, **pip** has served as the cornerstone of Python's packaging
- A lots of tools for Python modules / PVE manager: **poetry**, **virtualenv**, **venv**, **conda**...
- By 2025, **uv** became **pip** replacement:
  - ⇝ unified environment, dependency, and tool manager
  - ⇝ capable of handling every stage of the development lifecycle from interpreter installation to dependency synchronization and script execution.

# UV ⇝ the new Python project/PVE manager

## 😀 UV from Astral

**docs.astral.sh/uv/**

- For more than a decade, **pip** has served as the cornerstone of Python's packaging
- A lots of tools for Python modules / PVE manager: **poetry**, **virtualenv**, **venv**, **conda**...
- By 2025, **uv** became **pip** replacement:
  - ⇝ unified environment, dependency, and tool manager
  - ⇝ capable of handling every stage of the development lifecycle from interpreter installation to dependency synchronization and script execution.

# UV ⇝ the new Python project/PVE manager

## 😀 UV from Astral

**docs.astral.sh/uv/**

- For more than a decade, **pip** has served as the cornerstone of Python's packaging
- A lots of tools for Python modules / PVE manager: **poetry**, **virtualenv**, **venv**, **conda**...
- By 2025, **uv** became **pip** replacement:
  - ⇝ unified environment, dependency, and tool manager
  - ⇝ capable of handling every stage of the development lifecycle from interpreter installation to dependency synchronization and script execution.

# UV ⇝ the new Python project/PVE manager

## 😀 UV from Astral

**docs.astral.sh/uv/**

- For more than a decade, **pip** has served as the cornerstone of Python's packaging
- A lots of tools for Python modules / PVE manager: **poetry**, **virtualenv**, **venv**, **conda**...
- By 2025, **uv** became **pip** replacement:
  - ⇝ unified environment, dependency, and tool manager
  - ⇝ capable of handling every stage of the development lifecycle from interpreter installation to dependency synchronization and script execution.

# UV ⇝ the new Python project/PVE manager

## Installing UV

- **macOS & GNU/Linux**

  In a terminal window:

  ```
  curl -LsSf https://astral.sh/uv/install.sh | sh
  ```

- **Windows**

  In a PowerShell window:

  ```
  powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
  ```

# UV ⇝ the new Python project/PVE manager

## Command: `uv init [dir]`

- Create a project directory with a PVE inside:

```
uv init myproj
```

- Initialize a PVE within an existing directory:

```
cd myproj
uv init
```

# UV ⇝ the new Python project/PVE manager

## Command: `uv init [dir]`

- Create a project directory with a PVE inside:

```
uv init myproj
```

- Initialize a PVE within an existing directory:

```
cd myproj
uv init
```

Creates/uses the directory `myproj` and creates inside:

- `.gitignore` & `README.md` files and a ready to use `.git` sub-dir
- `pyproject.toml`: lists the version of the installed Python modules
- `.python-version`: the version of the PVE's interpreter
- `uv.lock`: detailed informations about modules dependencies
- as soon as necessary creates the `.venv` sub-directory to hold the PVE with the right version of the Python interpreter.

# UV ⇝ the new Python project/PVE manager

## Command: `uv init [dir]`

- Create a project directory with a PVE inside:

```
uv init myproj
```

- Initialize a PVE within an existing directory:

```
cd myproj
uv init
```

Creates/uses the directory `myproj` and creates inside:

- `.gitignore` & `README.md` files and a ready to use `.git` sub-dir
- `pyproject.toml`: lists the version of the installed Python modules
- `.python-version`: the version of the PVE's interpreter
- `uv.lock`: detailed informations about modules dependencies
- as soon as necessary creates the `.venv` sub-directory to hold the PVE with the right version of the Python interpreter.

# UV ⇝ the new Python project/PVE manager

## Command: `uv init [dir]`

- Create a project directory with a PVE inside:

```
uv init myproj
```

- Initialize a PVE within an existing directory:

```
cd myproj
uv init
```

Creates/uses the directory `myproj` and creates inside:

- `.gitignore` & `README.md` files and a ready to use `.git` sub-dir
- `pyproject.toml`: lists the version of the installed Python modules
- `.python-version`: the version of the PVE's interpreter
- `uv.lock`: detailed informations about modules dependencies
- as soon as necessary creates the `.venv` sub-directory to hold the PVE with the right version of the Python interpreter.

# UV ⇝ the new Python project/PVE manager

## Command: `uv init [dir]`

- Create a project directory with a PVE inside:

```
uv init myproj
```

- Initialize a PVE within an existing directory:

```
cd myproj
uv init
```

Creates/uses the directory `myproj` and creates inside:

- `.gitignore` & `README.md` files and a ready to use `.git` sub-dir
- `pyproject.toml`: lists the version of the installed Python modules
- `.python-version`: the version of the PVE's interpreter
- `uv.lock`: detailed informations about modules dependencies
- as soon as necessary creates the `.venv` sub-directory to hold the PVE with the right version of the Python interpreter.

# UV ⇝ the new Python project/PVE manager

## Command: `uv init [dir]`

- Create a project directory with a PVE inside:

```
uv init myproj
```

- Initialize a PVE within an existing directory:

```
cd myproj
uv init
```

Creates/uses the directory `myproj` and creates inside:

- `.gitignore` & `README.md` files and a ready to use `.git` sub-dir
- `pyproject.toml`: lists the version of the installed Python modules
- `.python-version`: the version of the PVE's interpreter
- `uv.lock`: detailed informations about modules dependencies
- as soon as necessary creates the `.venv` sub-directory to hold the PVE with the right version of the Python interpreter.

# UV ⇝ the new Python project/PVE manager

## Command: `uv init [dir]`

- Create a project directory with a PVE inside:

```
uv init myproj
```

- Initialize a PVE within an existing directory:

```
cd myproj
uv init
```

Creates/uses the directory `myproj` and creates inside:

- `.gitignore` & `README.md` files and a ready to use `.git` sub-dir
- `pyproject.toml`: lists the version of the installed Python modules
- `.python-version`: the version of the PVE's interpreter
- `uv.lock`: detailed informations about modules dependencies
- as soon as necessary creates the `.venv` sub-directory to hold the PVE with the right version of the Python interpreter.

# UV

Terminal

Terminal Terminal

```
jlc@kong:tmp$ uv init myproj
Initialized project `myproj` at `/home/jlc/tmp/myproj`
jlc@kong:tmp$ cd myproj
jlc@kong:myproj$ tree -a -L 1
.
├── .git
├── .gitignore
├── main.py
├── pyproject.toml
├── .python-version
└── README.md

2 directories, 5 files
jlc@kong:myproj$ cat .python-version
3.13
jlc@kong:myproj$ uv run python
Using CPython 3.13.12
Creating virtual environment at: .venv
Python 3.13.12 (main, Feb 12 2026, 00:45:41) [Clang 21.1.4 ] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
jlc@kong:myproj$
```

PVE with the right version of the Python interpreter.

# UV ⇝ the new Python project/PVE manager

**Command: `uv init [dir] --python` ⇝ `uv init [dir] -p`**

- To choose the version of the PVE's Python interpreter
- Create a project directory with a PVE inside:

```
uv init myproj -p 3.x
```

- Initialize a PVE within an existing directory:

```
cd myproj
uv init -p 3.x
```

# UV ⇝ the new Python project/PVE manager

**Command: `uv init [dir] --python` ⇝ `uv init [dir] -p`**

- To choose the version of the PVE's Python interpreter
- Create a project directory with a PVE inside:

```
uv init myproj -p 3.x
```

- Initialize a PVE within an existing directory:

```
cd myproj
uv init -p 3.x
```

- Creates/uses the directory `myproj` with all the stuff inside...
- As soon as necessary creates the `.venv` sub-directory to hold the PVE with the the desired version of the Python interpreter.

# UV

Terminal

Terminal | Terminal

```
jlc@kong:tmp$ uv init myproj -p 3.15
Initialized project `myproj` at `/home/jlc/tmp/myproj`
jlc@kong:tmp$ cd myproj
jlc@kong:myproj$ tree -a -L 1
.
├── .git
├── .gitignore
├── main.py
├── pyproject.toml
├── .python-version
└── README.md

2 directories, 5 files
jlc@kong:myproj$ cat .python-version
3.15
jlc@kong:myproj$ uv run python
Using CPython 3.15.0a6
Creating virtual environment at: .venv
Python 3.15.0a6 (main, Feb 12 2026, 00:43:06) [Clang 21.1.4 ] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
jlc@kong:myproj$
```

# UV ⇝ the new Python project/PVE manager

## Command: `uv add`

- To add a Python module to the PVE
- Let **uv** choose the last version of the module compatible with the interpreter version:

```
uv add module_name
```

- Choose manually a version of the module:

```
uv add module_name==version_number     # excatly this version
uv add module_name>=version_number     # at lest this version
uv add module_name>=n1,<n2             # between n1 and n2
```

# UV ⇝ the new Python project/PVE manager

## Command: `uv add`

- To add a Python module to the PVE
- Let **uv** choose the last version of the module compatible with the interpreter version:

```
uv add module_name
```

- Choose manually a version of the module:

```
uv add module_name==version_number     # excatly this version
uv add module_name>=version_number     # at lest this version
uv add module_name>=n1,<n2             # between n1 and n2
```

- **Very fast** (parallel) download of the module and its dependencies
- **Very efficient cache mechanism**
- Installs the module in the `.venv` directory
- Updates the file `pyproject.toml`
- Creates/updates `uv.locks` (exact version of the installed modules).

# UV ⇝ the new Python project/PVE manager

## Command: `uv add`

- To add a Python module to the PVE
- Let **uv** choose the last version of the module compatible with the interpreter version:

```
uv add module_name
```

- Choose manually a version of the module:

```
uv add module_name==version_number     # excatly this version
uv add module_name>=version_number     # at lest this version
uv add module_name>=n1,<n2             # between n1 and n2
```

- **Very fast** (parallel) download of the module and its dependencies
- **Very efficient cache mechanism**
- Installs the module in the `.venv` directory
- Updates the file `pyproject.toml`
- Creates/updates `uv.locks` (exact version of the installed modules).

# UV ⇝ the new Python project/PVE manager

## Command: `uv add`

- To add a Python module to the PVE
- Let **uv** choose the last version of the module compatible with the interpreter version:

```
uv add module_name
```

- Choose manually a version of the module:

```
uv add module_name==version_number     # excatly this version
uv add module_name>=version_number     # at lest this version
uv add module_name>=n1,<n2             # between n1 and n2
```

- **Very fast** (parallel) download of the module and its dependencies
- **Very efficient cache mechanism**
- Installs the module in the `.venv` directory
- Updates the file `pyproject.toml`
- Creates/updates `uv.locks` (exact version of the installed modules).

# UV ⇝ the new Python project/PVE manager

## Command: `uv remove`

- To remove a Python module to the PVE:

```
uv remove module_name
```

# UV ⇝ the new Python project/PVE manager

## Command: `uv remove`

- To remove a Python module to the PVE:

```
uv remove module_name
```

- Removes the module from the `.venv` directory
- Updates the files `pyproject.toml` and `uv.locks`.

# UV ⇝ the new Python project/PVE manager

## Command: `uv pip list`

- To list the Python modules loaded in the PVE

```
uv pip list
```

# UV ⇝ the new Python project/PVE manager

**Command: `uv pip list`**

- To list the Python modules loaded in the PVE

```
uv pip list
```

- lists the modules that have been installed in `.venv` directory

# UV ⇝ the new Python project/PVE manager

## Command: `uv run`

- To run a Python program within the PVE:

```
uv run python_file.py
```

- To run a command:

```
uv run command
```

- Tries to run the file `python_file.py` using the Python modules and interpreter from the PVE
- Tries to run a file `command.py` within the project PVE or others environments...

# UV ⇝ the new Python project/PVE manager

## Command: `uv run`

- To run a Python program within the PVE:

```
uv run python_file.py
```

- To run a command:

```
uv run command
```

- Tries to run the file `python_file.py` using the Python modules and interpreter from the PVE
- Tries to run a file `command.py` within the project PVE or others environments...

# UV ⇝ the new Python project/PVE manager

## Command: `uv run`

- To run a Python program within the PVE:

```
uv run python_file.py
```

- To run a command:

```
uv run command
```

- Tries to run the file `python_file.py` using the Python modules and interpreter from the PVE
- Tries to run a file `command.py` within the project PVE or others environments...

# UV ⇝ the new Python project/PVE manager

## Command: `uv run`

- To run a Python program within the PVE:

```
uv run python_file.py
```

- To run a command:

```
uv run command
```

- Tries to run the file `python_file.py` using the Python modules and interpreter from the PVE
- Tries to run a file `command.py` within the project PVE or others environments...

# UV ⇝ the new Python project/PVE manager

## Command: `uv run`

- To run a Python program within the PVE:

```
uv run python_file.py
```

- To run a command:

```
uv run command
```

- Tries to run the file `python_file.py` using the Python modules and interpreter from the PVE
- Tries to run a file `command.py` within the project PVE or others environments...
  Example: `uv run jupyter lab`

# UV ⇝ the new Python project/PVE manager

## Use case 1: Try some Python code within a PVE..

- Let say you found a tuto on Internet with a sample Python code like this one:
  CustomTkinter - A Complete Tutorial...
- you want to try the Python code **without breaking your system's or session's Python installation**...
- the **uv** solution:
  - ⇝ create a PVE inside a directory: `uv init uv-customTk`
  - ⇝ go to the created directory: `cd uv-customTk`
  - ⇝ add modules to the PVE: `uv add customtkinter`
  - ⇝ run the Python code within the PVE: `uv run main.py`
  - ⇝ play with the code... when finished, delete the directory or keep it.

# UV ⇝ the new Python project/PVE manager

## Use case 1: Try some Python code within a PVE..

- Let say you found a tuto on Internet with a sample Python code like this one:
  CustomTkinter - A Complete Tutorial...
- you want to try the Python code **without breaking your system's or session's Python installation**...
- the **uv** solution:
  - ⇝ create a PVE inside a directory: `uv init uv-customTk`
  - ⇝ go to the created directory: `cd uv-customTk`
  - ⇝ add modules to the PVE: `uv add customtkinter`
  - ⇝ run the Python code within the PVE: `uv run main.py`
  - ⇝ play with the code... when finished, delete the directory or keep it.

# UV ⇝ the new Python project

## Use case 1: try some P

- Let say you found a tuto o
  like this one:
  CustomTkinter - A Compl

- you want to try the Python
  **system's or session's P**

- the **uv** solution:
  - ⇝ create a PVE inside a c
  - ⇝ go to the created direct
  - ⇝ add modules to the PV
  - ⇝ run the Python code wi
  - ⇝ play with the code... when finished, delete the directory or keep it.

## Getting Started

### Installation

Before diving into CustomTkinter, you need to install it. CustomTkinter is available on PyPI and can be installed using pip:

```
pip install customtkinter
```

### First Application

Let's create a simple "Hello World" application to get started:

```
import customtkinter as ctk

# Set appearance mode and default color theme
ctk.set_appearance_mode("dark")
ctk.set_default_color_theme("blue")

# Create the main window
app = ctk.CTk()
app.title("Hello CustomTkinter")
app.geometry("400x200")

# Add a label
label = ctk.CTkLabel(app, text="Hello, CustomTkinter!", font=("Helvetica"
label.pack(pady=20)

# Add a button
button = ctk.CTkButton(app, text="Click Me", command=lambda: print("Butto
button.pack(pady=10)

# Run the application
app.mainloop()
```

# UV ⇝ the new Python project/PVE manager

## Use case 1: Try some Python code within a PVE..

- Let say you found a tuto on Internet with a sample Python code like this one:
  CustomTkinter - A Complete Tutorial...
- you want to try the Python code **without breaking your system's or session's Python installation**...
- the uv solution:
  - ⇝ create a PVE inside a directory: `uv init uv-customTk`
  - ⇝ go to the created directory: `cd uv-customTk`
  - ⇝ add modules to the PVE: `uv add customtkinter`
  - ⇝ run the Python code within the PVE: `uv run main.py`
  - ⇝ play with the code... when finished, delete the directory or keep it.

# UV ⇝ the new Python project/PVE manager

## Use case 1: Try some Python code within a PVE..

- Let say you found a tuto on Internet with a sample Python code like this one:
  CustomTkinter - A Complete Tutorial...
- you want to try the Python code **without breaking your system's or session's Python installation**...
- the **uv** solution:
  - ⇝ create a PVE inside a directory: `uv init uv-customTk`
  - ⇝ go to the created directory: `cd uv-customTk`
  - ⇝ add modules to the PVE: `uv add customtkinter`
  - ⇝ run the Python code within the PVE: `uv run main.py`
  - ⇝ play with the code... when finished, delete the directory or keep it.

# UV ⇝ the new Python project/PVE manager

## Use case 1: Try some Python code within a PVE..

- Let say you found a tuto on Internet with a sample Python code like this one:
  CustomTkinter - A Complete Tutorial...
- you want to try the Python code **without breaking your system's or session's Python installation**...
- the **uv** solution:
  - ⇝ create a PVE inside a directory: `uv init uv-customTk`
  - ⇝ go to the created directory: `cd uv-customTk`
  - ⇝ add modules to the PVE: `uv add customtkinter`
  - ⇝ run the Python code within the PVE: `uv run main.py`
  - ⇝ play with the code... when finished, delete the directory or keep it.

# UV ⇝ the new Python project/PVE manager

## Use case 1: Try some Python code within a PVE..

- Let say you found a tuto on Internet with a sample Python code like this one:
  CustomTkinter - A Complete Tutorial...
- you want to try the Python code **without breaking your system's or session's Python installation**...
- the **uv** solution:
  - ⇝ create a PVE inside a directory: `uv init uv-customTk`
  - ⇝ go to the created directory: `cd uv-customTk`
  - ⇝ add modules to the PVE: `uv add customtkinter`
  - ⇝ run the Python code within the PVE: `uv run main.py`
  - ⇝ play with the code... when finished, delete the directory or keep it.

# UV ⇝ the new Python project/PVE manager

## Use case 1: Try some Python code within a PVE..

- Let say you found a tuto on Internet with a sample Python code like this one:
  CustomTkinter - A Complete Tutorial...
- you want to try the Python code **without breaking your system's or session's Python installation**...
- the **uv** solution:
  - ⇝ create a PVE inside a directory: `uv init uv-customTk`
  - ⇝ go to the created directory: `cd uv-customTk`
  - ⇝ add modules to the PVE: `uv add customtkinter`
  - ⇝ run the Python code within the PVE: `uv run main.py`
  - ⇝ play with the code... when finished, delete the directory or keep it.

# UV ⇝ the new Python project/PVE manager

## Use case 1: Try some Python code within a PVE..

- Let say you found a tuto on Internet with a sample Python code like this one:
  CustomTkinter - A Complete Tutorial...
- you want to try the Python code **without breaking your system's or session's Python installation**...
- the **uv** solution:
  - ⇝ create a PVE inside a directory: `uv init uv-customTk`
  - ⇝ go to the created directory: `cd uv-customTk`
  - ⇝ add modules to the PVE: `uv add customtkinter`
  - ⇝ run the Python code within the PVE: `uv run main.py`
  - ⇝ play with the code... when finished, delete the directory or keep it.

# UV ⇝ the new Python project/PVE manager

## ⚙ Use case 2: Initialize a new Python Project within a PVE..

- For any new project (small or hudge) let **uv** manage the PVE
  - ⇝ Create the project directory: `uv init project_dir -p 3.12`
  - ⇝ Go to project directory: `cd project_dir`
  - ⇝ add modules to the PVE:
    ```
    uv add tensorflow jupyterlab matplotlib  # for example
    ```
  - ⇝ Edit some Python files or Jupyter Notebook files...
  - ⇝ run any Python code within the PVE: `uv run main.py`
  - ⇝ run Jupyter lab within the PVE: `uv run jupyter lab`

# UV ⇝ the new Python project/PVE manager

## Use case 2: Initialize a new Python Project within a PVE..

- For any new project (small or hudge) let **uv** manage the PVE
  - ⇝ Create the project directory: `uv init project_dir -p 3.12`
  - ⇝ Go to project directory: `cd project_dir`
  - ⇝ add modules to the PVE:
    `uv add tensorflow jupyterlab matplotlib # for example`
  - ⇝ Edit some Python files or Jupyter Notebook files...
  - ⇝ run any Python code within the PVE: `uv run main.py`
  - ⇝ run Jupyter lab within the PVE: `uv run jupyter lab`

# UV ⇝ the new Python project/PVE manager

## Use case 2: Initialize a new Python Project within a PVE..

- For any new project (small or hudge) let **uv** manage the PVE
  - ⇝ Create the project directory: `uv init project_dir -p 3.12`
  - ⇝ Go to project directory: `cd project_dir`
  - ⇝ add modules to the PVE:
    `uv add tensorflow jupyterlab matplotlib # for example`
  - ⇝ Edit some Python files or Jupyter Notebook files...
  - ⇝ run any Python code within the PVE: `uv run main.py`
  - ⇝ run Jupyter lab within the PVE: `uv run jupyter lab`

# UV ⇝ the new Python project/PVE manager

## Use case 2: Initialize a new Python Project within a PVE..

- For any new project (small or hudge) let **uv** manage the PVE
  - ⇝ Create the project directory: `uv init project_dir -p 3.12`
  - ⇝ Go to project directory: `cd project_dir`
  - ⇝ add modules to the PVE:
    `uv add tensorflow jupyterlab matplotlib # for example`
  - ⇝ Edit some Python files or Jupyter Notebook files...
  - ⇝ run any Python code within the PVE: `uv run main.py`
  - ⇝ run Jupyter lab within the PVE: `uv run jupyter lab`

# UV ⇝ the new Python project/PVE manager

## Use case 2: Initialize a new Python Project within a PVE..

- For any new project (small or hudge) let **uv** manage the PVE
  - ⇝ Create the project directory: `uv init project_dir -p 3.12`
  - ⇝ Go to project directory: `cd project_dir`
  - ⇝ add modules to the PVE:
    ```
    uv add tensorflow jupyterlab matplotlib  # for example
    ```
  - ⇝ Edit some Python files or Jupyter Notebook files...
  - ⇝ run any Python code within the PVE: `uv run main.py`
  - ⇝ run Jupyter lab within the PVE: `uv run jupyter lab`

# UV ⇝ the new Python project/PVE manager

## ⚙ Use case 2: Initialize a new Python Project within a PVE..

- For any new project (small or hudge) let **uv** manage the PVE
  - ⇝ Create the project directory: `uv init project_dir -p 3.12`
  - ⇝ Go to project directory: `cd project_dir`
  - ⇝ add modules to the PVE:
    ```
    uv add tensorflow jupyterlab matplotlib  # for example
    ```
  - ⇝ Edit some Python files or Jupyter Notebook files...
  - ⇝ run any Python code within the PVE: `uv run main.py`
  - ⇝ run Jupyter lab within the PVE: `uv run jupyter lab`

# UV ⇝ the new Python project/PVE manager

## Use case 2: Initialize a new Python Project within a PVE..

- For any new project (small or hudge) let **uv** manage the PVE
  - ⇝ Create the project directory: `uv init project_dir -p 3.12`
  - ⇝ Go to project directory: `cd project_dir`
  - ⇝ add modules to the PVE:
    `uv add tensorflow jupyterlab matplotlib  # for example`
  - ⇝ Edit some Python files or Jupyter Notebook files...
  - ⇝ run any Python code within the PVE: `uv run main.py`
  - ⇝ run Jupyter lab within the PVE: `uv run jupyter lab`

# UV ⇝ the new Python project/PVE manager

## Use case 2: Initialize a new Python Project within a PVE..

- For any new project (small or hudge) let **uv** manage the PVE
  - ⇝ Create the project directory: `uv init project_dir -p 3.12`
  - ⇝ Go to project directory: `cd project_dir`
  - ⇝ add modules to the PVE:
    ```
    uv add tensorflow jupyterlab matplotlib  # for example
    ```
  - ⇝ Edit some Python files or Jupyter Notebook files...
  - ⇝ run any Python code within the PVE: `uv run main.py`
  - ⇝ run Jupyter lab within the PVE: `uv run jupyter lab`

# UV ⇝ the new Python project/PVE manager

## Use case 3: Download a Git repository using uv

- Many Git Repositories use **uv** to manage their Python project
- For example: UV-Marimo_at_I2M
- Clone the repository (`git clone ....`)
  or download the ZIP
- Change Directory to the repo root and synchronize the PVE to dwoanload and install all the required Python modules:

```
uv sync
```

# UV ⇝ the new Python project/PVE manager

## Use case 3: Download a Git repository using uv

- Many Git Repositories use **uv** to manage their Python project
- For example: UV-Marimo_at_I2M
- Clone the repository (`git clone ....`)
  or download the ZIP
- Change Directory to the repo root and synchronize the PVE to dwoanload and install all the required Python modules:

```
uv sync
```

# UV ⇝ the new Python project/PVE manager

## Use case 3: Download a Git repository using uv

- Many Git Repositories use **uv** to manage their Python project
- For example: UV-Marimo_at_I2M
- Clone the repository (`git clone ....`) or download the ZIP
- Change Directory to the repo root and synchronize the PVE to dwoanload and install all the required Python modules:

```
uv sync
```

# UV ⇝ the new Python project/PVE manager

## Use case 3: Download a Git repository using uv

- Many Git Repositories use **uv** to manage their Python project
- For example: UV-Marimo_at_I2M
- Clone the repository (`git clone ....`)
  or download the ZIP
- Change Directory to the repo root and synchronize the PVE to dwoanload and install all the required Python modules:

```
uv sync
```

# UV ⇝ the new Python project/PVE manager

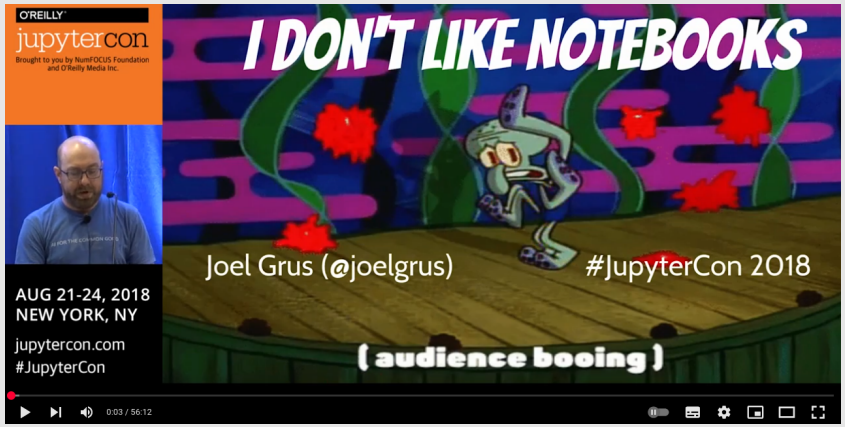
## Use case 3: Download a Git repository using uv

- Many Git Repositories use **uv** to manage their Python project
- For example: UV-Marimo_at_I2M
- Clone the repository (`git clone ....`)
  or download the ZIP
- Change Directory to the repo root and synchronize the PVE to dwoanload and install all the required Python modules:

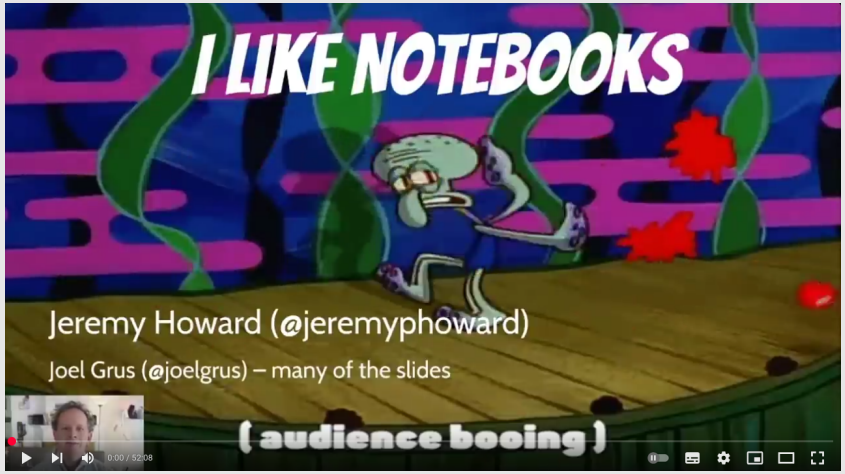
```
uv sync
```

# Jupyter notebooks *pros/cons*

some people don't like working with Jupyter notebooks... [Gru18]

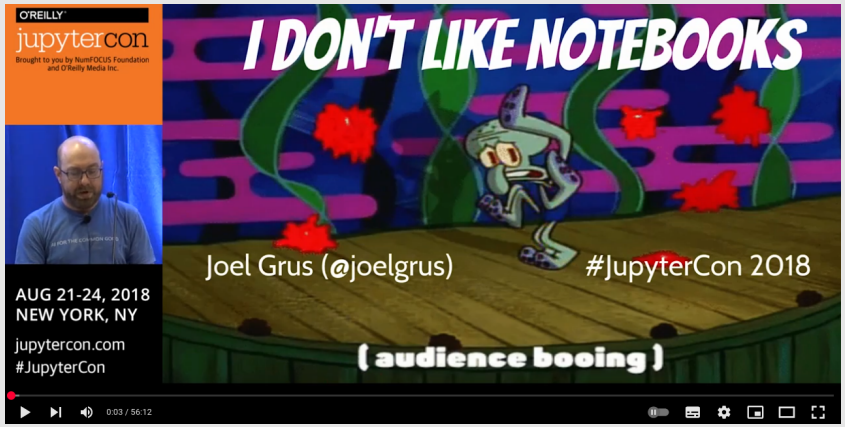



and some people do like... [How20]

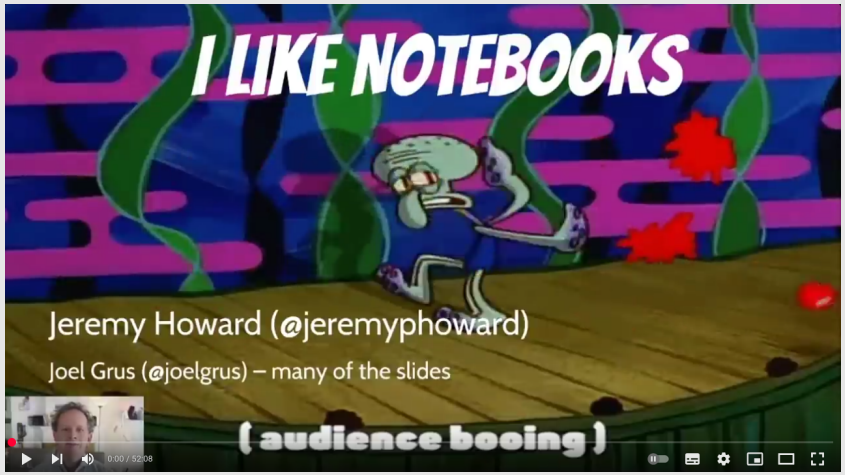


# Jupyter notebooks *pros/cons*

some people don't like working with Jupyter notebooks... [Gru18]




and some people do like... [How20]




🤔 What are the main *pros/cons* of Jupyter Notebooks...

# Jupyter notebooks *pros/cons*

## Some advantages

- Very well known...
- A very large number of published notebooks, covering very varied application areas
- Huge user community, a lot of tutos
- Powerfull to create documentation with embeded graphs, pictures...
- Nicely permissive: interactive asynchronous execution of cells
- and many more...

# Jupyter notebooks *pros/cons*

## Some advantages

- Very well known...
- A very large number of published notebooks, covering very varied application areas
- Huge user community, a lot of tutos
- Powerfull to create documentation with embeded graphs, pictures...
- Nicely permissive: interactive asynchronous execution of cells
- and many more...

# Jupyter notebooks *pros/cons*

## 😀 Some advantages

- Very well known...
- A very large number of published notebooks, covering very varied application areas
- Huge user community, a lot of tutos
- Powerfull to create documentation with embeded graphs, pictures...
- Nicely permissive: interactive asynchronous execution of cells
- and many more...

# Jupyter notebooks *pros/cons*

## 😀 Some advantages

- Very well known...
- A very large number of published notebooks, covering very varied application areas
- Huge user community, a lot of tutos
- Powerfull to create documentation with embeded graphs, pictures...
- Nicely permissive: interactive asynchronous execution of cells
- and many more...

# Jupyter notebooks *pros/cons*

## 😀 Some advantages

- Very well known...
- A very large number of published notebooks, covering very varied application areas
- Huge user community, a lot of tutos
- Powerfull to create documentation with embeded graphs, pictures...
- Nicely permissive: interactive asynchronous execution of cells
- and many more...

# Jupyter notebooks *pros/cons*

## 😀 Some advantages

- Very well known...
- A very large number of published notebooks, covering very varied application areas
- Huge user community, a lot of tutos
- Powerfull to create documentation with embeded graphs, pictures...
- Nicely permissive: interactive asynchronous execution of cells 🤪
- and many more...

# Jupyter notebooks *pros/cons*

## 😀 Some advantages

- Very well known...
- A very large number of published notebooks, covering very varied application areas
- Huge user community, a lot of tutos
- Powerfull to create documentation with embeded graphs, pictures...
- Nicely permissive: interactive asynchronous execution of cells 🤪
- and many more... 🤩

# Jupyter notebooks *pros/cons*

## Some desadvantages...

- **Hidden state** if you delete a cell, its variables stay in memory, which other cells may still reference ⇝ reproducibility issues.
- **Lack of reproducibility** the code you see doesn't necessarily match the outputs on the page or the program state.
- One study [Guz20] analyzed 10 million Jupyter notebooks and found that 36% of them weren't reproducible.
- Notebooks JSON format embeds binary outputs (plots, images...) ⇝ Git versionning is dificult
- and even more...

# Jupyter notebooks *pros/cons*

## Some desadvantages...

- **Hidden state** if you delete a cell, its variables stay in memory, which other cells may still reference $\rightsquigarrow$ reproducibility issues.
- **Lack of reproducibility** the code you see doesn't necessarily match the outputs on the page or the program state.
- One study [Guz20] analyzed 10 million Jupyter notebooks and found that 36% of them weren't reproducible.
- Notebooks JSON format embeds binary outputs (plots, images...) $\rightsquigarrow$ Git versionning is dificult
- and even more...

# Jupyter notebooks *pros/cons*

## Some desadvantages...

- **Hidden state** if you delete a cell, its variables stay in memory, which other cells may still reference ⇝ reproducibility issues.
- **Lack of reproducibility** the code you see doesn't necessarily match the outputs on the page or the program state.
- One study [Guz20] analyzed 10 million Jupyter notebooks and found that 36% of them weren't reproducible.
- Notebooks JSON format embeds binary outputs (plots, images...) ⇝ Git versionning is dificult
- and even more...

# Jupyter notebooks *pros/cons*

## Some desadvantages...

- **Hidden state** if you delete a cell, its variables stay in memory, which other cells may still reference ⇝ reproducibility issues.
- **Lack of reproducibility** the code you see doesn't necessarily match the outputs on the page or the program state.
- One study [Guz20] analyzed 10 million Jupyter notebooks and found that 36% of them weren't reproducible.
- Notebooks JSON format embeds binary outputs (plots, images...) ⇝ Git versionning is dificult
- and even more...

# Jupyter notebooks *pros/cons*

## Some desadvantages...

- **Hidden state** if you delete a cell, its variables stay in memory, which other cells may still reference $\rightsquigarrow$ reproducibility issues.
- **Lack of reproducibility** the code you see doesn't necessarily match the outputs on the page or the program state.
- One study [Guz20] analyzed 10 million Jupyter notebooks and found that 36% of them weren't reproducible.
- Notebooks JSON format embeds binary outputs (plots, images...) $\rightsquigarrow$ Git versionning is dificult
- and even more...

# Jupyter notebooks *pros/cons*

## 🙁 Some desadvantages...

- **Hidden state** if you delete a cell, its variables stay in memory, which other cells may still reference ⇝ reproducibility issues.
- **Lack of reproducibility** the code you see doesn't necessarily match the outputs on the page or the program state.
- One study [Guz20] analyzed 10 million Jupyter notebooks and found that 36% of them weren't reproducible.
- Notebooks JSON format embeds binary outputs (plots, images...) ⇝ Git versionning is dificult
- and even more... 🥴

# Marimo notebooks: pure Python

docs.marimo.io

*"marimo notebook: a reinvention of the Jupyter notebook as a reproducible, reactive, interactive, and shareable Python program that can be executed as scripts or deployed as interactive web apps"*

# Marimo notebooks: pure Python

docs.marimo.io

*"marimo notebook: a reinvention of the Jupyter notebook as a reproducible, reactive, interactive, and shareable Python program that can be executed as scripts or deployed as interactive web apps"*

## Advantages...

- Only pure Python code in a marimo notebook!
  - ⇝ Cell outputs are not stored in the notebook
- Cells dependency is defined as a DAG (Directed Acyclic Graph)
  - ⇝ At notebook startup all cells are executed to give the outputs, in the order of their dependencies, not the order they appear.

# Marimo notebooks: pure Python

docs.marimo.io

*"marimo notebook: a reinvention of the Jupyter notebook as a reproducible, reactive, interactive, and shareable Python program that can be executed as scripts or deployed as interactive web apps"*

## Advantages...

- Only pure Python code in a marimo notebook!
  - ⇝ Cell outputs are not stored in the notebook
- Cells dependency is defined as a DAG (Directed Acyclic Graph)
  - ⇝ At notebook startup all cells are executed to give the outputs, in the order of their dependencies, not the order they appear.

# Marimo notebooks: pure Python

Notebooks/simple.py

Simple marimo notebook to show the content of the Python file

```
1 import numpy as np
```

```
1 x = np.arange(1,5)
```

```
array([1, 2, 3, 4])
```

```
1 x
```

```
1 import marimo as mo
```

PYTHON | MARKDOWN | SQL | GENERATE WITH AI

*ion of the*
*ble,*
*ble Python*
*s scripts or*
*s"*

cyclic Graph)

ne outputs,

ey appear.

# Marimo notebooks: pure Py

Notebooks/sim

Simple marimo notebook to show the content of the Py

```
import numpy as np
```

```
x = np.arange(1,5)
```

```
array([1, 2, 3, 4])
```

```
x
```

```
import marimo as mo
```

PYTHON MARKDOWN

```python
import marimo

__generated_with = "0.13.15"
app = marimo.App(width="medium")


@app.cell(hide_code=True)
def _(mo):
    mo.md(r"""Simple marimo notebook to show the content of the Python file
    return


@app.cell
def _():
    import numpy as np
    return (np,)


@app.cell
def _(np):
    x = np.arange(1,5)
    return (x,)


@app.cell
def _(x):
    x
    return


@app.cell
def _():
    import marimo as mo
    return (mo,)


if __name__ == "__main__":
    app.run()
```

# Marimo notebooks: pure Python

docs.marimo.io

*"marimo notebook: a reinvention of the Jupyter notebook as a reproducible, reactive, interactive, and shareable Python program that can be executed as scripts or deployed as interactive web apps"*

## Advantages...

- Only pure Python code in a marimo notebook!
  - ⇝ Cell outputs are not stored in the notebook
- Cells dependency is defined as a DAG (Directed Acyclic Graph)
  - ⇝ At notebook startup all cells are executed to give the outputs, in the order of their dependencies, not the order they appear.

# Marimo notebooks: pure Python

docs.marimo.io

*"marimo notebook: a reinvention of the Jupyter notebook as a reproducible, reactive, interactive, and shareable Python program that can be executed as scripts or deployed as interactive web apps"*

## Advantages...

- Only pure Python code in a marimo notebook!
  - ⇝ Cell outputs are not stored in the notebook
- Cells dependency is defined as a DAG (Directed Acyclic Graph)
  - ⇝ At notebook startup all cells are executed to give the outputs, in the order of their dependencies, not the order they appear.

# Marimo notebooks: pure Python

docs.marimo.io

*"marimo notebook: a reinvention of the Jupyter notebook as a reproducible, reactive, interactive, and shareable Python program that can be executed as scripts or deployed as interactive web apps"*

## Advantages...

- Only pure Python code in a marimo notebook!
  - ⇝ Cell outputs are not stored in the notebook
- Cells dependency is defined as a DAG (Directed Acyclic Graph)
  - ⇝ At notebook startup all cells are executed to give the outputs, in the order of their dependencies, not the order they appear.

# Marimo notebooks: pure Python

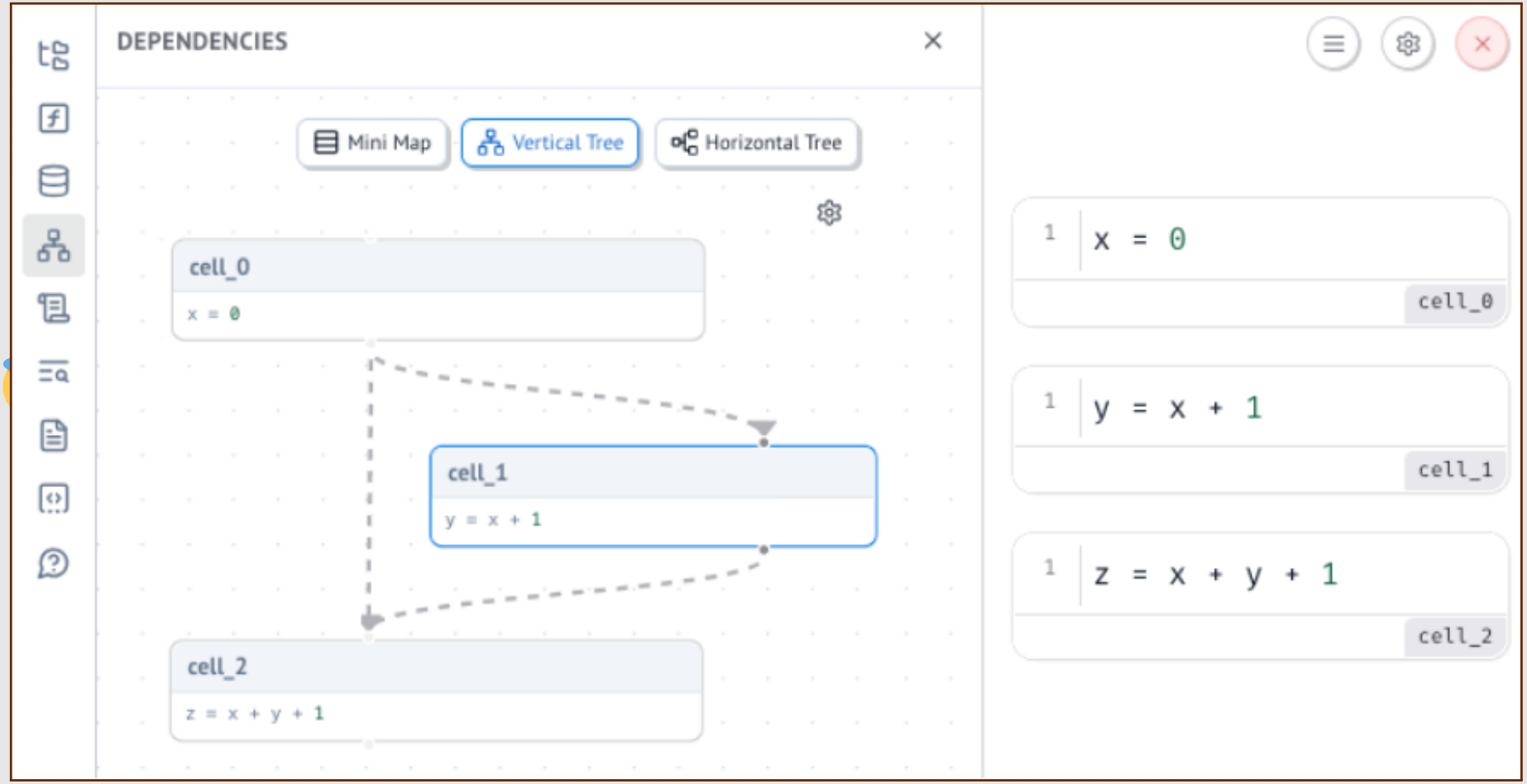



image from [Vey24]

# Marimo notebooks: pure Python

docs.marimo.io

*"marimo notebook: a reinvention of the Jupyter notebook as a reproducible, reactive, interactive, and shareable Python program that can be executed as scripts or deployed as interactive web apps"*

## Advantages...

- Only pure Python code in a marimo notebook!
  - ⇝ Cell outputs are not stored in the notebook
- Cells dependency is defined as a DAG (Directed Acyclic Graph)
  - ⇝ At notebook startup all cells are executed to give the outputs, in the order of their dependencies, not the order they appear.

# Marimo notebooks: pure Python

## Advantages...

- **reactive**
  - ⇝ run a cell: marimo reactively runs all dependent cells or marks them as *stale*.
  - ⇝ delete a cell: marimo scrubs its variables from program memory, no hidden state.
- **Consistent state**: notebook code, outputs, and program state are guaranted to be consistent.
- **Reproducible**: no hidden state, deterministic execution. Notebooks output is always up to date.

# Marimo notebooks: pure Python

## Advantages...

- **reactive**
  - ⇝ run a cell: marimo reactively runs all dependent cells or marks them as *stale*.
  - ⇝ delete a cell: marimo scrubs its variables from program memory, no hidden state.
- **Consistent state**: notebook code, outputs, and program state are guaranted to be consistent.
- **Reproducible**: no hidden state, deterministic execution. Notebooks output is always up to date.

# Marimo notebooks: pure Python

## Advantages...

- **reactive**
  - ⇝ run a cell: marimo reactively runs all dependent cells or marks them as *stale*.
  - ⇝ delete a cell: marimo scrubs its variables from program memory, no hidden state.
- **Consistent state**: notebook code, outputs, and program state are guaranted to be consistent.
- **Reproducible**: no hidden state, deterministic execution. Notebooks output is always up to date.

# Marimo notebooks: pure Python

## Advantages...

- **reactive**
  - ⇝ run a cell: marimo reactively runs all dependent cells or marks them as *stale*.
  - ⇝ delete a cell: marimo scrubs its variables from program memory, no hidden state.
- **Consistent state**: notebook code, outputs, and program state are guaranted to be consistent.
- **Reproducible**: no hidden state, deterministic execution. Notebooks output is always up to date.

# Marimo notebooks: pure Python

## Advantages...

- **reactive**
  - ⇝ run a cell: marimo reactively runs all dependent cells or marks them as *stale*.
  - ⇝ delete a cell: marimo scrubs its variables from program memory, no hidden state.
- **Consistent state**: notebook code, outputs, and program state are guaranted to be consistent.
- **Reproducible**: no hidden state, deterministic execution. Notebooks output is always up to date.

# Marimo notebooks: pure Python

## Advantages...

- Rich library of UI elements:
  - ⇝ Input UI elements: slider, date, number, dropdown, file brower, butto, check button, radio button...
  - ⇝ Layout UI elements: tab, array, accordion, vstack, hstack...
- UI elements are automatically synchronized with the notebook code:
  - ⇝ Interacting with a UI element bound to a global variable automatically runs all cells that reference it.
- and many more...

# Marimo notebooks: pure Python

## Advantages...

- Rich library of UI elements:
  - ⇝ Input UI elements: slider, date, number, dropdown, file brower, butto, check button, radio button...
  - ⇝ Layout UI elements: tab, array, accordion, vstack, hstack...
- UI elements are automatically synchronized with the notebook code:
  - ⇝ Interacting with a UI element bound to a global variable automatically runs all cells that reference it.
- and many more...

# Marimo notebooks: pure Python

## Advantages...

- Rich library of UI elements:
  - ⇝ Input UI elements: slider, date, number, dropdown, file brower, butto, check button, radio button...
  - ⇝ Layout UI elements: tab, array, accordion, vstack, hstack...
- UI elements are automatically synchronized with the notebook code:
  - ⇝ Interacting with a UI element bound to a global variable automatically runs all cells that reference it.
- and many more...

# Marimo notebooks: pure Python

## Advantages...

- Rich library of UI elements:
  - ⇝ Input UI elements: slider, date, number, dropdown, file brower, butto, check button, radio button...
  - ⇝ Layout UI elements: tab, array, accordion, vstack, hstack...
- UI elements are automatically synchronized with the notebook code:
  - ⇝ Interacting with a UI element bound to a global variable automatically runs all cells that reference it.
- and many more...

# Marimo notebooks: pure Python

## Advantages...

- Rich library of UI elements:
  - ⇝ Input UI elements: slider, date, number, dropdown, file brower, butto, check button, radio button...
  - ⇝ Layout UI elements: tab, array, accordion, vstack, hstack...
- UI elements are automatically synchronized with the notebook code:
  - ⇝ Interacting with a UI element bound to a global variable automatically runs all cells that reference it.
- and many more...

# Marimo notebooks: pure Python

## disadvantages...

- A variable (object) cannot be defined twice in different cells
  - ⇝ use the format `_name` to make the variable local to the cell
  - ⇝ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
  - ⇝ The entire notebook is run at startup...
- At startup automatic run of CPU-hungry cells can be problematic:
  - ⇝ *user setting* ▶ disable notebook autorun at startup (*lazy* notebook)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
  - ⇝ *user setting* ▶ disable cell auto run on cell change (*lazy cell*): marimo will just mark affected cells as *stale*

# Marimo notebooks: pure Python

## disadvantages...

- A variable (object) cannot be defined twice in different cells
  - ⇝ use the format `_name` to make the variable local to the cell
  - ⇝ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
  - ⇝ The entire notebook is run at startup...
- At startup automatic run of CPU-hungry cells can be problematic:
  - ⇝ *user setting* ▶ disable notebook autorun at startup (*lazy* notebook)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
  - ⇝ *user setting* ▶ disable cell auto run on cell change (*lazy cell*): marimo will just mark affected cells as *stale*

# Marimo notebooks: pure Python

## disadvantages...

- A variable (object) cannot be defined twice in different cells
  - ⇝ use the format `_name` to make the variable local to the cell
  - ⇝ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
  - ⇝ The entire notebook is run at startup...
- At startup automatic run of CPU-hungry cells can be problematic:
  - ⇝ *user setting* ▶ disable notebook autorun at startup (*lazy* notebook)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
  - ⇝ *user setting* ▶ disable cell auto run on cell change (*lazy cell*): marimo will just mark affected cells as *stale*

# Marimo notebooks: pure Python

## disadvantages...

- A variable (object) cannot be defined twice in different cells
  - ⇝ use the format `_name` to make the variable local to the cell
  - ⇝ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
  - ⇝ The entire notebook is run at startup...
- At startup automatic run of CPU-hungry cells can be problematic:
  - ⇝ *user setting* ▶ disable notebook autorun at startup (*lazy* notebook)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
  - ⇝ *user setting* ▶ disable cell auto run on cell change (*lazy cell*): marimo will just mark affected cells as *stale*

# Marimo notebooks: pure Python

## disadvantages...

- A variable (object) cannot be defined twice in different cells
  - ⇝ use the format `_name` to make the variable local to the cell
  - ⇝ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
  - ⇝ The entire notebook is run at startup...
- At startup automatic run of CPU-hungry cells can be problematic:
  - ⇝ *user setting* ▶ disable notebook autorun at startup (*lazy* notebook)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
  - ⇝ *user setting* ▶ disable cell auto run on cell change (*lazy cell*): marimo will just mark affected cells as *stale*

# Marimo notebooks: pure Python

## disadvantages...

- A variable (object) cannot be defined twice in different cells
  - ⇝ use the format `_name` to make the variable local to the cell
  - ⇝ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
  - ⇝ The entire notebook is run at startup...
- At startup automatic run of CPU-hungry cells can be problematic:
  - ⇝ *user setting* ▶ disable notebook autorun at startup (*lazy* notebook)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
  - ⇝ *user setting* ▶ disable cell auto run on cell change (*lazy cell*): marimo will just mark affected cells as *stale*

# Marimo notebooks: pure Python

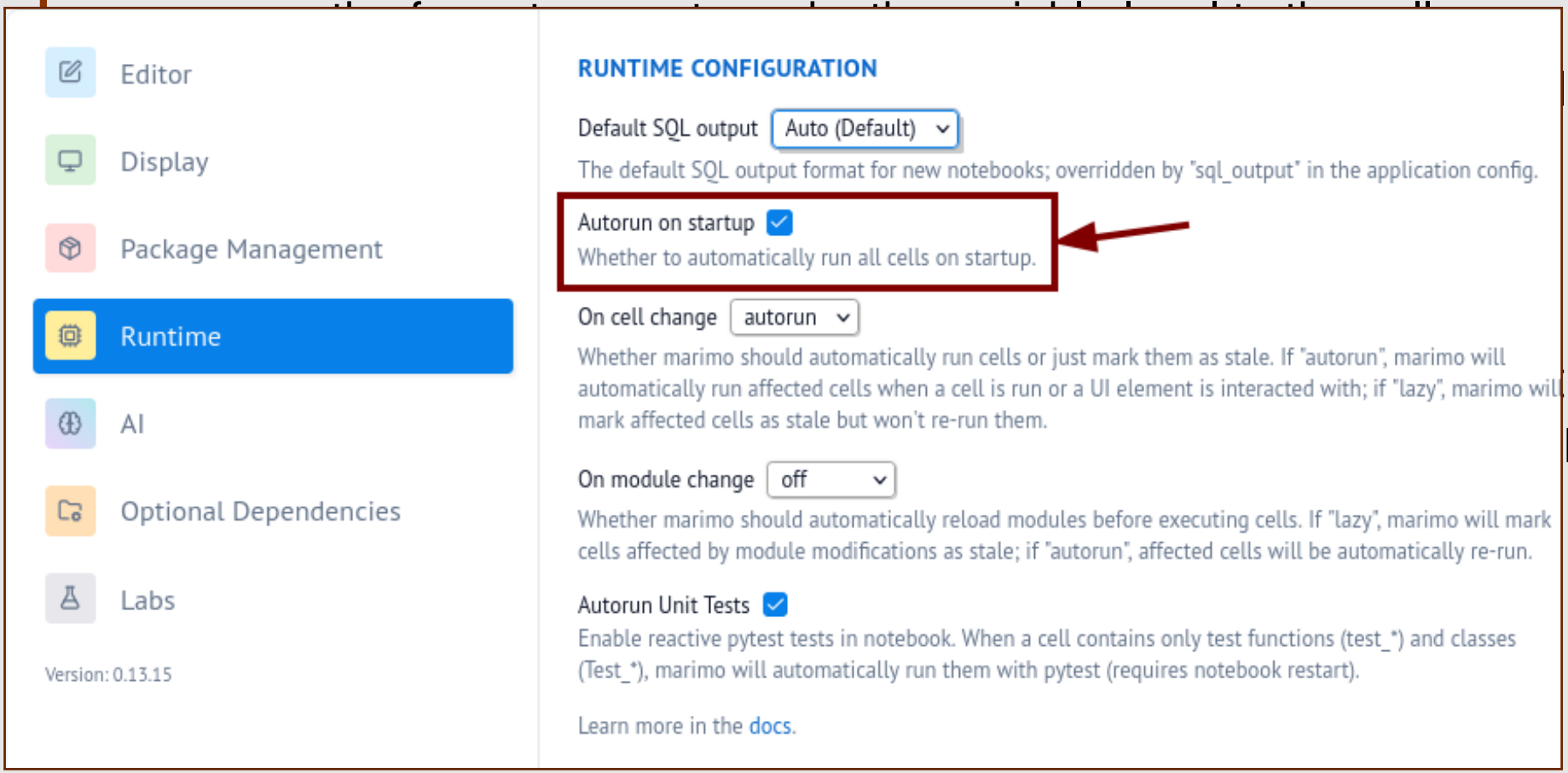
## disadvantages...

- A variable (object) cannot be defined twice in different cells
  - ⇝ use the format `_name` to make the variable local to the cell
  - ⇝ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
  - ⇝ The entire notebook is run at startup...
- At startup automatic run of CPU-hungry cells can be problematic:
  - ⇝ *user setting* ▶ disable notebook autorun at startup (*lazy* notebook)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
  - ⇝ *user setting* ▶ disable cell auto run on cell change (*lazy cell*): marimo will just mark affected cells as *stale*

# Marimo notebooks: pure Python

## disadvantages...

- A variable (object) cannot be defined twice in different cells



# Marimo notebooks: pure Python

## disadvantages...

- A variable (object) cannot be defined twice in different cells
  - ⇝ use the format `_name` to make the variable local to the cell
  - ⇝ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
  - ⇝ The entire notebook is run at startup...
- At startup automatic run of CPU-hungry cells can be problematic:
  - ⇝ *user setting* ▶ disable notebook autorun at startup (*lazy* notebook)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
  - ⇝ *user setting* ▶ disable cell auto run on cell change (*lazy cell*): marimo will just mark affected cells as *stale*

# Marimo notebooks: pure Python

## disadvantages...

- A variable (object) cannot be defined twice in different cells
  - ⇝ use the format `_name` to make the variable local to the cell
  - ⇝ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
  - ⇝ The entire notebook is run at startup...
- At startup automatic run of CPU-hungry cells can be problematic:
  - ⇝ *user setting* ▶ disable notebook autorun at startup (*lazy* notebook)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
  - ⇝ *user setting* ▶ disable cell auto run on cell change (*lazy cell*): marimo will just mark affected cells as *stale*

# Marimo notebooks: pure Python

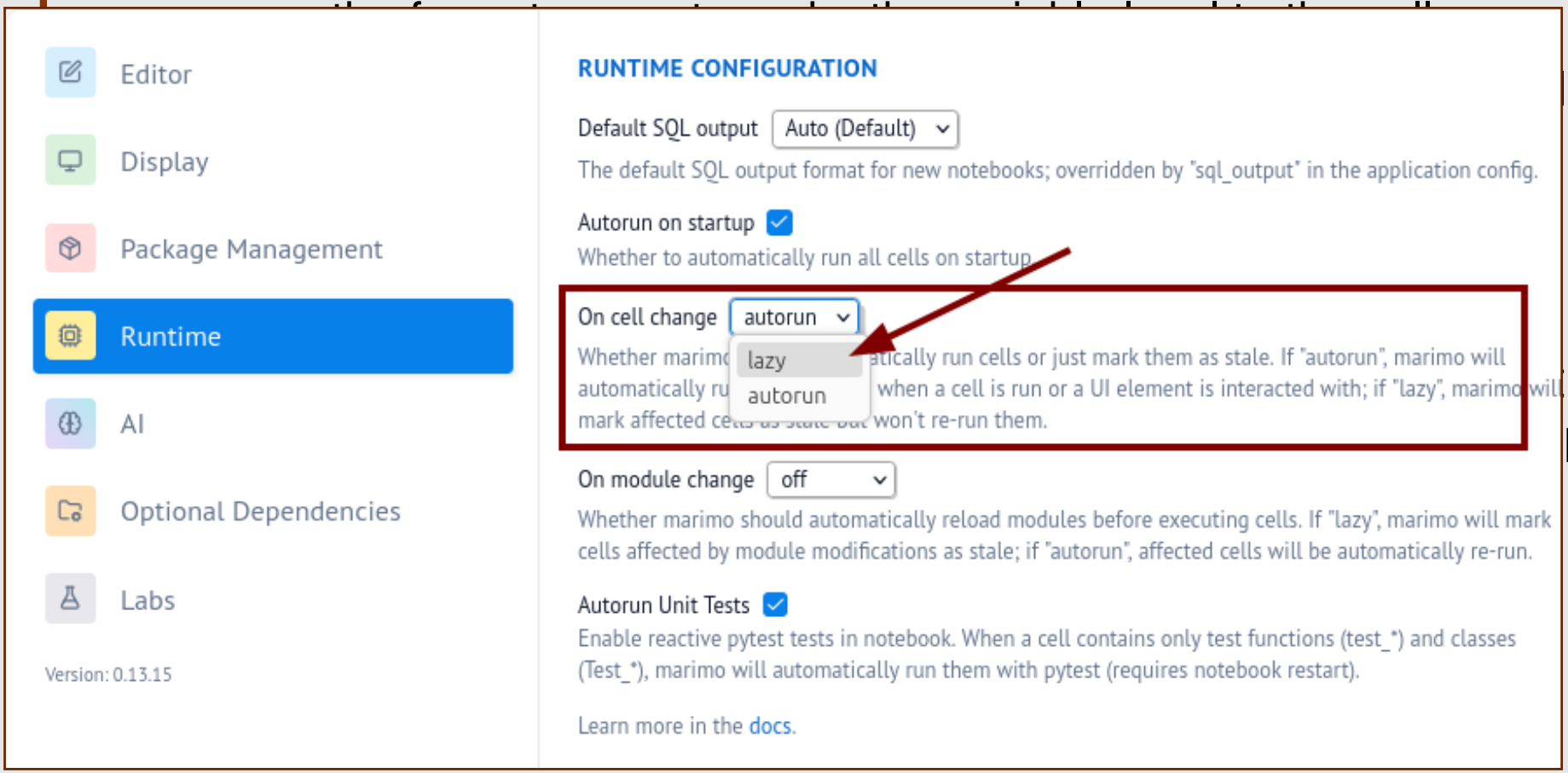
## disadvantages...

- A variable (object) cannot be defined twice in different cells
  - ⇝ use the format `_name` to make the variable local to the cell
  - ⇝ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
  - ⇝ The entire notebook is run at startup...
- At startup automatic run of CPU-hungry cells can be problematic:
  - ⇝ *user setting* ▶ disable notebook autorun at startup (*lazy* notebook)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
  - ⇝ *user setting* ▶ disable cell auto run on cell change (*lazy cell*): marimo will just mark affected cells as *stale*

# Marimo notebooks: pure Python

## disadvantages...

- A variable (object) cannot be defined twice in different cells




ic:
k)

# Marimo notebooks: pure Python

## disadvantages...

- A variable (object) cannot be defined twice in different cells
  - ⇝ use the format `_name` to make the variable local to the cell
  - ⇝ or encapsulate object definitions in functions to make them local
- Cell outputs is not written in the notebook (pure Python...)
  - ⇝ The entire notebook is run at startup...
- At startup automatic run of CPU-hungry cells can be problematic:
  - ⇝ *user setting* ▶ disable notebook autorun at startup (*lazy* notebook)
- Updating a cell or UI widget triggers the automatic execution of all dependent cells:
  - ⇝ *user setting* ▶ disable cell auto run on cell change (*lazy cell*): marimo will just mark affected cells as *stale*

## Marimo installation & run

- marimo installation with uv (see docs.astral.sh/uv/)

```
uv add marimo        # installs the marimo module within the virt. env.
uv run marimo edit   # runs marimo editor within your favorite web browser
```

- marimo installation with pip

```
pip install marimo
marimo edit
```

- marimo installation with conda (universal virtualenv)

```
conda install -c conda-forge marimo
marimo edit
```

## Marimo notebooks examples

- `1-simple.py`
- `2-reactive.py`
- `3-interactive.py`
- `4-interactive-plot.py`
- `5-ML.py`
- `5-ML_diskcache.py`

# Bibliography I

## Jupyter notebooks pros/cons...

[Gru18] J. Grus, "I don't like notebook," Allen Institute for Artificial Intelligence. (2018), https://www.youtube.com/watch?v=7jiPeIFXb6U (cit. on pp. 73, 74).

[Guz20] A. Guzharina, "We downloaded 10,000,000 jupyter notebooks from github – this is what we learned," (2020), https://blog.jetbrains.com/datalore/2020/12/17/we-downloaded-10-000-000-jupyter-notebooks-from-github-this-is-what-we-learned/#consistency-of-notebooks (cit. on pp. 82–87).

[How20] J. Howard, "I like notebook," fast.ai. (2020), https://www.youtube.com/watch?v=9Q6sLbz37gk (cit. on pp. 73, 74).

## Marimo Notebooks

[Agr24] A. Agrawal, "Lessons learned reinventing the python notebook," marimo.io. (May 8, 2024), https://marimo.io/blog/lessons-learned?ref=blog.ippon.fr.

[Vey24] S. Vey, "Marimo, ou l'avenir du notebook," ippon positive technology. (June 26, 2024), https://blog.ippon.fr/2024/06/26/marimo-ou-l-avenir-du-notebook/ (cit. on p. 96).

# Bibliography II

## UV


[Dha25] DhanushKumar, "Uv in python: The fastest package & project manager (complete guide + example project)," Medium. (Nov 18, 2025), https://blog.stackademic.com/uv-in-python-the-fastest-package-project-manager-complete-guide-example-project-91e6a6048e80.

[Ram25] L. P. Ramos, "Managing python projects with uv: An all-in-one solution)," https://realpython.com. (Apr 28, 2025), https://realpython.com/python-uv/.

[Ray25] D. Ray, "From pip to uv: A modern revolution in python package management," Medium. (Nov 11, 2025), https://medium.com/@debisreer/from-pip-to-uv-a-modern-revolution-in-python-package-management-62dd8ac91df2.

[Rob26] S. Robert, "Uv python : Remplacer pip et poetry (uvx, lockfile, docker)," https://blog.stephane-robert.info. (Jan 19, 2026), https://blog.stephane-robert.info/docs/developper/programmation/python/uv/.

[Tuy25] B. Tuychiev, "Python uv : Le guide ultime du gestionnaire de paquets python le plus rapide," Datacamp.com. (Jan 9, 2025), https://www.datacamp.com/fr/tutorial/python-uv.